

PROJEKT 2: BOOKSCOUT

Von Webdaten zu einem interaktiven Buchanalyse-Dashboard

Web Scraping • Data Processing • Analytics • Interactive Dashboard

AGENDA

- Einleitung, Teamaufteilung, Organisation
- Problembeschreibung und Erkennung der Aufgabe
- Aufbau der IT-Infrastruktur (Backend/Frontend)
- Visualisierung des Dashboards
- Datenanalyse und Erkenntnis
- Schlusswort/Fragerunde

EINLEITUNG, TEAMAUFTeilUNG, ORGANISATION

- Entwicklung eines Dashboards (gescrapte Buchdaten) inkl. Features und Datenanalyse
- Team-Members: Ben, Andre, Nataliia, Marzieh, Christin
- Programmierung & Aufsetzen des Dashboards jedes Mitgliedes sowie Vorstellung
 - Gewinner-Dashboard als Hauptkonzept genutzt (MVP vorhanden)
 - Optimierung des Dashboards und Features-Engineering (Besprechung)
 - Aufteilung in Programmierungsgruppe (Ben, Andre, Marzieh) und Präsentationsgruppe (Nataliia, Christin)
- Individuelle Pausenregelung, Kommunikation & Dateiaustausch über teams
- Abstimmungsprinzip: „Mehrheit entscheidet“
- Arbeitsprinzip: „Jeder nach seinen Stärken“

PROBLEMBESCHREIBUNG & ERKENNUNG DER AUFGABE

- aktuelle Webseite (<https://books.toscrape.com>) unübersichtlich, unsortiert, schlecht filterbar & nicht auswertbar, keine Datentransparenz und Qualität
- BookScout (End-to-End Data Pipeline) löst Problem mangelnder Markttransparenz durch datengetriebene Analyse von Preisstrukturen, Bewertungsmustern und Value-Segmenten
 - Verbindung von Data Engineering mit interaktiver BI-Analyse
 - Anbindung mehrerer API-Scraping Plattformen möglich
 - Direktkauf der Bücher über Link möglich

AUFBAU DER IT-INFRASTRUKTUR (BACKEND/FRONTEND)

```
books.toscrape.com  
(Externe Webquelle - HTTP, HTML)  
↓  
BookScraper (scraper.py)  
(Python 3, requests, BeautifulSoup)  
↓  
OpenLibrary API (api_client.py)  
(REST-API, JSON, requests)  
↓  
main.py (Pipeline-Orchestrierung & Datenanreicherung)  
(Python Application Layer - Kontrolllogik & Upsert-Management)  
↓  
SQLite-Datenbank (bookscout.db)  
(File-based Relationale Datenbank)  
↓  
SQLAlchemy ORM (database.py + models.py)  
(ORM-Schicht - Datenmodellierung & Session-Management)  
↓  
DataProcessor (Filter- & Transformationslogik)  
(Pandas - DataFrame-Verarbeitung & Business-Logik)  
↓  
Analytics Layer (analytics_features.py - Segmentierung & Value Score)  
(Feature Engineering - Quantilberechnung, Kennzahlen, Aggregationen)  
↓  
Streamlit Frontend (app.py - UI, Interaktion & Visualisierung)  
(Web Framework - Streamlit + Altair (Vega-Lite Engine))
```

DATENERFASSUNG (WEB SCRAPING)

Bookscraper (scraper.py) - primäre Datenquelle der Pipeline

- Datenquelle:
 - books.toscrape.com
- Technologien:
 - Python, requests, BeautifulSoup (Web Scraping)
- Scraping in zwei Schritten:
 - Übersichtsseiten: Titel, Preis, Rating
 - Detailseiten pro Buch: Kategorie, Beschreibung, UPC, Cover-URL

```
items = soup.select(["article.product_pod"])
print(f"[Scraper]   Bücher auf Seite: {len(items)}")

for item in items:
    title = item.h3.a.get("title", "").strip()

    price_tag = item.select_one(".price_color")
    price_text = price_tag.text.strip() if price_tag else ""
    price = self._parse_price(price_text)
    if price is None:
        print(f"[Scraper]   ⚠ Preis nicht parsebar: {repr(price_text)}")
        continue

    rating_tag = item.select_one("p.star-rating")
    rating_classes = rating_tag.get("class", []) if rating_tag else []
```

API-BASIERTE DATENANREICHERUNG

OpenLibrary API (`api_client.py`) - Erweiterung der Buchdaten

- Technologien:
 - REST-API, JSON, requests
- Strategie der Datenanreicherung:
 - Suche über **UPC**
 - Falls Treffer →
 - Metadaten ergänzt
 - Falls kein Treffer →
 - **Fallback über Titel-Suche**
- Zusätzliche Metadaten:
 - Autor
 - Veröffentlichungsjahr

```
# Final: Kombiniert (ISBN -> Titel)
# -----
def enrich(self, title: str, isbn_or_upc: str | None = None) -> dict:
    """Kombinierte Strategie:

    1) Erst ISBN/UPC über /api/books probieren
    2) Wenn nichts gefunden -> Titel-Suche als Fallback
    """

    if isbn_or_upc:
        extra = self.enrich_by_isbn(isbn_or_upc)
        # Wenn wir schon ein Cover haben, reicht es meistens
        if extra.get("cover_url") or extra.get("author") or extra.get("publish_year"):
            return extra

    return self.enrich_by_title(title)
```

PIPELINE-ORCHESTRIERUNG

main.py - Steuerung der Data Pipeline

- Ablauf der Verarbeitung:
 1. Web Scraping der Buchdaten
 2. API-Anreicherung (OpenLibrary)
 3. Datenvalidierung
 4. Speicherung in der Datenbank
 5. Aktualisierung der Preis-Historie

→ Ergebnis: End-to-End Data Pipeline

```
scraper = BookScraper()
api = OpenLibraryClient(sleep_sec=0.3, debug=False)

print("Starte Scraping...")
books = scraper.scrape_all_books(sleep_sec=0.0)
print(f"{len(books)} Bücher gescraped.")

enriched = []
for i, b in enumerate(books, start=1):
    title = b.get("title", "")
    upc = b.get("upc", "")

    try:
        extra = api.enrich(title=title, isbn_or_upc=upc)
        # Cover nicht überschreiben (kommt aus Scraping)
        extra.pop("cover_url", None)
        b.update(extra)
```

DATENBANKVERBINDUNG

database.py

- Nutzung einer lokalen SQLite-Datenbank
- Verbindung wird über SQLAlchemy ORM hergestellt
- Dynamische Bestimmung des Datenbankpfads
- Engine verwaltet die Verbindung
- SessionLocal führt Datenbankoperationen aus
- Base dient als Grundlage für alle Datenbankmodelle

```
1  from pathlib import Path
2  from sqlalchemy import create_engine
3  from sqlalchemy.ext.declarative import declarative_base
4  from sqlalchemy.orm import sessionmaker
5
6  # Absoluter DB-Pfad (immer im Projektordner neben database.py)
7  BASE_DIR = Path(__file__).resolve().parent
8  DB_PATH = BASE_DIR / "bookscout.db"
9
10 SQLALCHEMY_DATABASE_URL = f"sqlite:///{{DB_PATH.as_posix()}}"
11
12 engine = create_engine(SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False})
13 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
14 Base = declarative_base()
```

DATENBANKMODELLE

models.py

Haupttabelle: Books

Speichert Titel, Kategorie, Preis, Rating und Verfügbarkeit

Zusätzliche Felder: Autor, Erscheinungsjahr, Cover

Weitere Tabelle: PriceHistory

Speichert Preisänderungen über die Zeit

damit können Preisverläufe über mehrere Scans analysiert werden

```
1 from datetime import datetime
2
3 from sqlalchemy import Column, DateTime, Float, Integer, String, Text
4
5 from database import Base
6
7
8 Windsurf: Refactor | Explain
9 class Book(Base):
10     __tablename__ = "books"
11
12     id = Column(Integer, primary_key=True, index=True)
13
14     # Listing-Daten
15     title = Column(String, index=True)
16     category = Column(String, index=True)
17     price = Column(Float)
18     rating = Column(Integer) # 1..5
19     availability = Column(String)
20
21     # Detailseiten-Daten
22     upc = Column(String, index=True)
23     description = Column(Text)
24
25     # API-Anreicherung (Open Library - best effort)
26     author = Column(String, nullable=True)
27     publish_year = Column(Integer, nullable=True)
28     cover_url = Column(String, nullable=True)
29
30 Windsurf: Refactor | Explain
31 class PriceHistory(Base):
32     __tablename__ = "price_history"
33
34     id = Column(Integer, primary_key=True)
35
36     # UPC als Referenz auf das Buch
37     upc = Column(String, index=True)
38
39     # Gespeicherter Preis zum Zeitpunkt des Scraping
40     price = Column(Float)
41
42     # Zeitpunkt der Erfassung
43     scraped_at = Column(DateTime, default=datetime.utcnow)
```

VORBEREITUNG DER DATEN

data_processor.py

Klasse: DataProcessor

- Konvertiert Daten in ein pandas DataFrame
- filter_df() ermöglicht flexible Filter
- Filter nach Kategorie, Preis, Rating oder Titel

```
Windsurf: Refactor | Explain
class DataProcessor:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def to_dataframe(self, books: list) -> pd.DataFrame:
        df = pd.DataFrame(books)
        for col in ("price", "rating"):
            if col in df.columns:
                df[col] = pd.to_numeric(df[col], errors="coerce")
        return df

    Windsurf: Refactor | Explain | Generate Docstring | X
    def filter_df(self, df: pd.DataFrame, categories: list, price_min: float, price_max: float, ratings: list, title_query: str):
        out = df.copy()
        if categories:
            out = out[out["category"].isin(categories)]

        out = out[out["price"].between(price_min, price_max)]

        if ratings:
            out = out[out["rating"].isin(ratings)]

        if title_query:
            out = out[out["title"].str.contains(title_query, case=False, na=False)]

        return out

    Windsurf: Refactor | Explain | Generate Docstring | X
    def price_stats_by_category(self, df: pd.DataFrame) -> pd.DataFrame:
        return (
            df.groupby("category")["price"]
            .agg(["mean", "min", "max"])
            .reset_index()
            .sort_values("mean", ascending=False)
        )
```

ANALYSEFUNKTIONEN

analytics_features.py

Implementierte Funktionen (Analysen) im System:

- Datenqualität:
data_quality_metrics()
- Autorenanalyse:
author_analytics()
- Segmentierung von Büchern:
segment_books()
- Preis-Leistungs-Analyse:
value_score()

APP.PY

Die Schaltzentrale der Webanwendung

BIBLIOTHEKEN & IMPORTS

```
# Externe Bibliotheken

import pandas as pd

import streamlit as st

import altair as alt

from urllib.parse import quote_plus


# Eigene Module

from database import SessionLocal, Base, engine
from models import Book
from data_processor import DataProcessor
from main import run_scrape
from analytics_features import (
    data_quality_metrics, author_analytics,
    segment_books, value_score )
```



Buchdaten als Tabelle im Speicher (DataFrame)



Baut die Weboberfläche: Buttons, Tabs, Charts



Interaktive Diagramme mit wenig Code



Kodiert Sonderzeichen in Amazon-URLs korrekt

DATEN LADEN & CACHING

```
@st.cache_data(show_spinner=False)
def load_books_df_cached():
    Base.metadata.create_all(bind=engine)

    db = SessionLocal()
    try:
        rows = db.query(Book).all()
        df = pd.DataFrame([
            {"title": r.title,
             "price": r.price, ...}
            for r in rows ])
        return df
    finally:
        db.close()
```

@st.cache_data

Ergebnis wird gespeichert – beim nächsten Aufruf werden die Daten NICHT erneut aus der DB geladen. Spart Zeit und Ressourcen.

List Comprehension

Jedes Buch-Objekt (r) wird in ein Dictionary umgewandelt: {"title": r.title, "price": r.price, ...} – daraus entsteht der DataFrame.

try / finally

Die Datenbankverbindung wird IMMER geschlossen – egal ob ein Fehler auftritt oder nicht. Verhindert offene Verbindungen.

REFRESH-BUTTON & DATENLADEN

```
colA, colB = st.columns([1, 5])
with colA:
    if st.button('🔄 Refresh'):
        with st.spinner(
            'Scraping...'):
            run_scrape()
            st.cache_data.clear()
            st.success('Data updated!')
            st.rerun()

# Daten laden
df = load_books_df_cached()

if df.empty:
    st.warning('No data. Click Refresh.')
    st.stop()
```

1 Scraper starten
run_scrape() aus dem main-Modul wird aufgerufen – er holt neue Buchdaten von der Website.

2 Cache leeren
st.cache_data.clear() löscht die gespeicherten Daten, damit beim nächsten Laden die frischen Daten aus der DB kommen.

3 Seite neu laden
st.rerun() lädt die gesamte App neu – der Nutzer sieht sofort die aktualisierten Bücher.

4 Sicherheitsnetz
Falls die DB leer ist (z. B. nach erstem Start), zeigt st.stop() eine Warnung und hält die App an.

SIDEBAR – FILTERFORMULAR

```
st.sidebar.header('Filters')

with st.sidebar.form('filter_form'):
    st.multiselect('Category',
                   options=all_categories,
                   key='f_categories')
    st.multiselect('Rating (stars)',
                   options=all_ratings,
                   key='f_ratings')
    st.slider('Price range (£)',
              min_value=min_price,
              max_value=max_price,
              key='f_price_range')
    st.form_submit_button('✅ Apply')
st.sidebar.text_input('Search title',
                     key='f_title_query')
```

st.sidebar.form

Bündelt alle Filter – Änderungen werden erst beim Klick auf 'Apply' übernommen. Verhindert zu viele Neuladen.

st.multiselect

Ermöglicht Mehrfachauswahl von Kategorien und Bewertungen (1–5 Sterne).

st.slider

Preisbereich-Filter: Schieberegler zwischen dem günstigsten und teuersten Buch in der Datenbank.

text_input außerhalb

Die Titelsuche liegt bewusst außerhalb des Formulars – so sucht sie direkt ohne 'Apply'-Klick.

DIE 4 TABS DER APP

```
tab_overview, tab_analyses, tab_books, tab_compare = st.tabs(["Overview", "Analyses", "Books", "Compare"])
```

01
Overview

02
Analyses

03
Books

04
Compare

TAB-INHALTE & VERWENDETE FUNKTIONEN

Overview	Analyses	Books	Compare
<code>kpi_block(filtered)</code>	<code>data_quality_metrics(df)</code>	<code>show_book_detail(df, upc)</code>	<code>kpi_block(df_a) / kpi_block(df_b)</code>
<code>rating_distribution_chart(f iltered)</code>	<code>author_analytics(filtered)</code>	<code>toggle_favorite(upc)</code>	<code>rating_distribution_chart(d f_a/b)</code>
<code>st.altair_chart(price_chart)</code>	<code>segment_chart_and_drilldown (filtered)</code>	<code>amazon_search_url(title, author)</code>	<code>alt.Chart(cmp_df).mark_boxp lot()</code>
<code>export_csv_bytes(filtered)</code>	<code>value_score(filtered)</code>	<code>st.session_state['favorites ']</code>	<code>pd.concat([df_a, df_b])</code>
Alle Analysefunktionen (data_quality_metrics, author_analytics, segment_books, value_score) kommen aus analytics_features.py			

HILFSFUNKTIONEN IN app.py

export_csv_bytes(df)

```
return df.to_csv(index=False)
        .encode("utf-8")
```

Wandelt DataFrame in downloadbare CSV-Datei um

_stars_html_from_value(rating)

```
filled = "★" * int(round(rating))
empty  = "★" * (5 - int(round(rating)))
```

Erzeugt HTML-Sterne ★★☆☆☆ für die Bewertungsanzeige

amazon_search_url(title, author)

```
q = f"{title} {author}"
return "amazon.com/s?k=" + quote_plus(q)
```

Baut fertigen Amazon-Suchlink mit URL-kodierten Sonderzeichen

kpi_block(df)

```
c1.metric('Books', len(df))
c2.metric('Avg price', df['price'].mean())
```

Zeigt drei KPI-Kacheln nebeneinander: Anzahl, Ø Preis, Ø Bewertung

SESSION STATE

💡 Was ist Session State?

Streamlit lädt die gesamte Seite bei jedem Klick neu – alle normalen Variablen werden dabei zurückgesetzt. `st.session_state` ist ein Dictionary, das Werte über mehrere Klicks und Seitenneuladen hinweg speichert. Es funktioniert wie ein Kurzzeitgedächtnis der App.

🔄 Problem ohne Session State

Jeder Klick lädt die Seite neu. Ohne Speicher würden z. B. alle Eingabefelder sofort zurückgesetzt.

💾 Lösung: `st.session_state`

Werte wie die Eingabefelder werden im `session_state` abgelegt und bleiben während der gesamten Sitzung erhalten.

`mean()`

app.py – ZUSAMMENFASSUNG



Imports: 4 externe Bibliotheken + 5 eigene Module



4 Tabs: Overview, Analyses, Books, Compare



Daten aus SQLite-Datenbank mit
`@st.cache_data`



Hilfsfunktionen: CSV, Sterne, Amazon-Links,
KPIs



Sidebar mit 4 Filtern + Refresh-Button



Session State: Favoriten & Detail-Ansicht

LEARNINGS/SCHLUSSWORT

Lessons Learned & Conclusion

- **Von Webdaten zu strukturierten Analysen**

Wir haben gelernt, unstrukturierte Webdaten zu sammeln, zu bereinigen und in eine strukturierte Datenbank zu überführen.

- **Data Pipeline praktisch umgesetzt**

Das Projekt verbindet Web Scraping, API-Integration, Datenbank und Analytics in einem durchgehenden Workflow.

- **Daten verständlich visualisieren**

Mit einem interaktiven Dashboard können Nutzer Daten filtern, vergleichen und schnell wichtige Insights erkennen.

- **Basis für weitere Analysen**

Die strukturierte Datenbasis ermöglicht zukünftige Erweiterungen wie Preisanalysen oder weitere Datenquellen.

LEARNINGS/SCHLUSSWORT

Teamwork & Collaboration

- **Effektive Teamarbeit**

Wir haben gelernt, Aufgaben im Team zu koordinieren und gemeinsam auf ein klares Ziel hinzuarbeiten.

- **Gemeinsame Entscheidungen treffen**

Viele technische Lösungen waren möglich – wir mussten Optionen diskutieren und gemeinsam die beste Lösung wählen.

- **Flexibilität und Problemlösung**

Während der Entwicklung sind neue Herausforderungen entstanden, auf die wir schnell reagieren und Lösungen finden mussten.

- **Stärken im Team nutzen**

Jedes Teammitglied konnte seine individuellen Fähigkeiten einbringen, sodass wir effizient zusammenarbeiten und ein gemeinsames Ergebnis erreichen konnten.

NEXT STEPS

- **Preis-Trend-Analyse**

Analyse der Preisentwicklung einzelner Bücher sowie Vergleich von Durchschnittspreisen zwischen Kategorien über mehrere Zeitpunkte.

- **Einfache Buchempfehlungen**

Automatische Identifikation von Büchern mit hoher Bewertung, attraktivem Preis oder besonders guten Bewertungen innerhalb einer Kategorie.

- **Integration mehrerer Datenquellen**

Erweiterung der App zu einem Buch-Aggregator, der Daten von mehreren Websites sammelt, Preise vergleicht und direkte Links zu den Anbietern.

- **Nutzerfeedback und Bugfixing**

Sammlung von Nutzerfeedback sowie Identifikation und Behebung von Bugs zur Verbesserung von Stabilität und Benutzerfreundlichkeit.



**VIELEN DANK FÜR EURE
AUFMERKSAMKEIT**

FRAGEN?